

INSTITUTE OF ENGINEERING & MANAGEMENT

Department of Computer Science and Engineering
(Artificial Intelligence & Machine Learning)
IEM Newtown Campus

ASSIGNMENT 1

Lab Report: Core Fundamentals & Locators in Selenium

Name:	Roshni Ghosh Tagore
Enrollment No:	12023002028175
Course:	Python for Automation
Academic Year:	2023–2027
Department:	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
Campus:	IEM Newtown

Module Name: Tier 1: Core Fundamentals & Locators

Experiment Title: The Multi-Locator Challenge (Assignment 1)

1. Problem Statement

Automated web testing requires precise element identification and interaction across web pages. The task requires navigating to a login page (e.g., SauceDemo) and interacting with various form elements using three specific Selenium locator strategies:

- Find and interact with the `username` field using `By.ID`.
- Find and interact with the `password` field using `By.NAME`.
- Find and click the `login button` using `By.XPATH`.
- Validate that the resulting page URL contains `/inventory.html` after logging in.

2. Objective

- To gain hands-on experience in basic browser automation using Python and Selenium WebDriver.
- To practice element identification using distinct locator strategies (`By.ID` , `By.NAME` , `By.XPATH`).
- To implement assertion techniques for validating application state post-navigation.
- To ensure clean resource management using `try...finally` blocks.

3. Tools, Software, and Concepts Used

- **Programming Language:** Python
- **Automation Library:** Selenium WebDriver (`selenium.webdriver`)
- **IDE / Editor:** Visual Studio Code (VS Code)
- **Target Web Application:** SauceDemo (`https://www.saucedemo.com/`)
- **Key Concepts:** WebDriver initialization, DOM locators (ID, Name, XPath), user actions (`send_keys` , `click`), URL validation via assertion, driver lifecycle management (`quit()`).

4. Source Code / Implementation Steps

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    # 1. Navigate to target URL
```

```

driver.get("https://www.saucedemo.com/")
driver.maximize_window()

# 2. Interact using By.ID, By.NAME, and By.XPATH
driver.find_element(By.ID, "user-name").send_keys("standard_user")
driver.find_element(By.NAME, "password").send_keys("secret_sauce")
driver.find_element(By.XPATH, "//input[@id='login-button']").click()

# 3. Validation
current_url = driver.current_url
assert "/inventory.html" in current_url, f"Expected '/inventory.html' in URL, got {current_url}"
print("Assignment 1 Passed: Login successful and URL validated.")

finally:
    driver.quit()

```

5. Output with Clearly Labelled Screenshots

```

assignment1.py > ...
1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3
4  driver = webdriver.Chrome()
5
6  try:
7      # 1. Navigate to target URL
8      driver.get("https://www.saucedemo.com/")
9      driver.maximize_window()
10
11     # 2. Interact using By.ID, By.NAME, and By.XPATH
12     driver.find_element(By.ID, "user-name").send_keys("standard_user")
13     driver.find_element(By.NAME, "password").send_keys("secret_sauce")
14     driver.find_element(By.XPATH, "//input[@id='login-button']").click()
15
16     # 3. Validation
17     current_url = driver.current_url
18     assert "/inventory.html" in current_url, f"Expected '/inventory.html' in URL, got {current_url}"
19     print("Assignment 1 Passed: Login successful and URL validated.")
20
21 finally:
22     driver.quit()

```

Screenshot 1: Python Source Code Implementation in Visual Studio Code (assignment1.py)

```

(.venv) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots> python -u "c:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots\assignment1.py"
Assignment 1 Passed: Login successful and URL validated.
(.venv) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots>

```

Screenshot 2: Terminal Output Execution demonstrating successful execution and assertion passing

6. Brief Result, Observation, and Conclusion

Result:

The automation script was executed successfully in the terminal. The browser navigated to the SauceDemo website, logged in with the credentials, and verified that the target URL contained `/inventory.html`, printing: `"Assignment 1 Passed: Login successful and URL validated."`

Observation:

- `By.ID` provides a fast and unique method to target elements when standard IDs are present.
- `By.NAME` effectively targets form elements when ID attributes are absent or non-unique.
- `By.XPATH` allows flexible target selection based on element tags and specific attribute-value pairs.
- The post-login URL check accurately validates whether authentication succeeded before proceeding further.

Conclusion:

The experiment was successfully conducted. The fundamentals of Selenium WebDriver locators, browser interaction, assertion-based validation, and clean script teardown were thoroughly demonstrated and implemented.